SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED (CO4): Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool: Industry Problem Solving Task (Medium) *Assessment Tool Weightage: 35%* *Total Marks: 50*

Code of Conduct

I S LAKSHMI PRIYA (192524284) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S LAKSHMI PRIYA

Name: S LAKSHMI PRIYA **Reg. No: 192524284**

Question 1: File Organization Strategy for a 10-Million-Product E-Commerce Catalog

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Problem Understanding:

An e-commerce catalog must support: (a) fast point lookups when a customer opens a specific product page (search by product_id), (b) range/sorted browsing (price ranges, alphabetical listing, category filters), and (c) reasonably frequent inserts/updates as new products are added and stock/price changes. A single access pattern will not dominate, so the file organization must balance search speed, range-query support, and insert cost.

Recommended Strategy: Indexed-Sequential (B+ Tree Indexed) File Organization

A primary B+ Tree index clustered on product_id is recommended as the main file organization, with secondary (non-clustering) B+ Tree indexes on category and price to support catalog browsing and filtering. This is preferred over a pure heap, pure sorted, or pure hashed file because:

- Heap file: O(1) insert but O(n) search — unacceptable for 10 million records where every product-page view would require scanning up to 5 million blocks on average.

Cost Analysis (assumptions: 10,000,000 records, ~200-byte record, 4KB block ⇒ blocking factor ≈ 20 records/block ⇒ ≈ 500,000 data blocks; B+ Tree fan-out F ≈ 200):

File Organization	Point Search Cost (I/Os)	Range Query Cost	Insert Cost (I/Os)
Heap File	≈ 250,000 (n/2 blocks avg.)	O(n) full scan	≈ 1 (append only)
Sorted File	≈ 19 ($\log_2$ 500,000)	O(log n + k) — good	≈ 250,000 (avg. shift)
Static Hashing	≈ 1–2 (O(1) avg.)	Not supported (full scan)	≈ 1–2 (O(1) avg.)
B+ Tree Indexed (Recommended)	≈ 3–4 (logF n, F≈200)	O(logF n + k) via leaf links	≈ 3–4 (logF n, occasional split)

Justification:

The B+ Tree indexed-sequential organization reduces point-lookup cost from hundreds of thousands of I/Os (heap) to only 3–4 I/Os while still keeping insert cost in the same low single-digit range — something neither a pure sorted file nor pure hashing can achieve simultaneously. The high fan-out of B+ Tree nodes (hundreds of keys per 4KB block) keeps the tree shallow (only 3–4 levels for 10 million records), and leaf-level linking directly supports the category/price-range browsing that is central to an e-commerce UI. In practice this maps to a clustered index on product_id plus secondary non-clustering B+ Tree indexes on category and price, exactly as implemented in production RDBMS/e-commerce platforms.

Question 2: Indexing Strategy for a Banking Transaction System

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Problem Understanding:

The dominant query pattern in banking is: 'get all transactions for account X between date D1 and D2' (e.g., generating a monthly statement). This is a composite predicate — an equality condition on account_number combined with a range condition on transaction_date — not two independent single-column queries.

Recommended Indexing Strategy:

- Primary clustering index: a composite B+ Tree index on (account_number, transaction_date). Because it is clustering, all transactions for the same account are physically stored together on disk and, within an account, sorted by date. This means a statement query touches only the small number of contiguous blocks holding that account's transactions, and the date range filter is satisfied by simply scanning forward along the leaf chain — no extra I/O for the range condition.

Justification:

A composite clustering B+ Tree on (account_number, transaction_date) is the correct primary structure because the two most common predicates in the query (account + date range) are handled together by ONE index traversal: descend the tree to the account_number prefix (O(log n)), then walk the linked leaves for the date range (O(k) for k matching rows) — with zero extra disk seeks compared to running the filters separately. A single-column index on account_number alone would still require scanning all of that account's transactions to apply the date filter, which is unnecessary if the composite key is used. The secondary date-only index protects the less frequent but still important bank-wide reporting queries without slowing down the primary account-scoped workload.

Question 3: Multi-Level Indexing for a Healthcare Patient Records System

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Problem Understanding:

PatientID is the unique primary key used for direct record access (opening a specific patient's chart), while doctor_name is a non-unique attribute used to list all patients under a given doctor's care. These are two structurally different index requirements: a unique primary index and a non-unique secondary index, both of which should themselves be multi-level to keep even the index scan fast as the hospital's records grow.

Recommended Multi-Level Design:

- Level 0 (data file): patient records stored in a clustered file physically ordered by PatientID.

Justification:

Making both indexes multi-level (rather than a single flat index level) matters because a flat dense index on millions of patient records would itself span thousands of blocks, turning the 'index lookup' into another O(n) scan. By recursively indexing the index (multi-level / B+ Tree structure), the top level always fits in one memory-resident block, so a search for a PatientID or a doctor's patient list resolves in only 2–3 disk I/Os regardless of hospital size. Keeping the secondary doctor_name index separate and non-clustering (pointer-based) allows patient records to remain physically clustered by PatientID for fast chart retrieval, while doctor-based case-load queries still perform efficiently — satisfying both access patterns without duplicating patient data.

Question 4: B-Tree vs. B+ Tree for a Logistics Shipment Tracking System

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Problem Understanding:

A logistics system continuously creates new shipment records (pickup booked), updates their status, and eventually purges old/delivered shipments — a high-churn, insert/delete-heavy workload. It also commonly needs range queries: 'all shipments scheduled for delivery this week,' or 'all shipments on route R sorted by date.'

Criterion	B-Tree	B+ Tree
Data location	Keys + data stored at every node (internal and leaf)	Data stored only at leaf level; internal nodes hold routing keys only
Fan-out per node	Lower (node also carries data pointers, so fewer keys fit per 4KB block)	Higher (internal nodes carry only keys, so more keys fit per block ⇒ shallower tree)
Range queries	Requires in-order traversal across the tree — no leaf linking	Fast — leaf nodes are linked; range scan is a simple forward walk
Delete complexity	Can require merging/borrowing at any level, including internal nodes holding data — more complex rebalancing	Deletion only manipulates leaf nodes (data level); internal keys are just routing copies, simplifying rebalancing
Insert/split behaviour	Splits can occur at internal levels carrying data, slightly more overhead per split	Splits are simpler since leaves are uniform data containers; internal-node splits only copy a routing key upward

Recommendation: B+ Tree

The B+ Tree is clearly more suitable for this logistics scenario for three combined reasons: (1) higher fan-out from key-only internal nodes keeps the tree shallower, so both inserts and deletes touch fewer disk blocks even under heavy, continuous churn; (2) deletions are simpler and localized to the leaf level, which matters because shipment records are frequently deleted/archived once delivered; and (3) the linked-leaf structure directly supports the 'list of shipments by date/route' range queries that a logistics dashboard needs — a query a B-Tree cannot answer without a full in-order traversal. This is precisely why virtually every production RDBMS and file system index (e.g., InnoDB, NTFS) uses a B+ Tree rather than a classic B-Tree.

Question 5: Static vs. Extendible Hashing for a 5,000-Employee HR System

Bloom's Level: BL3 – Apply | Marks: 5

Question:

Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Problem Understanding:

The HR system needs fast point lookup of an employee record by employee_id (e.g., payroll processing, ID-card lookups). The dataset size (5,000 employees) is small, well-known in advance, and grows slowly and predictably (attrition/hiring), unlike a system with unpredictable, rapid growth.

Proposed Static Hashing Design:

Choose a bucket capacity of, say, 40 records/block (a typical employee record of ~100 bytes fits comfortably many-per-4KB block). To keep the load factor around 70–75% (avoiding excessive chaining), provision buckets for roughly 7,000 employee slots: number_of_buckets = ceil(7000 / 40) ≈ 175 buckets, using h(employee_id) = employee_id mod 175, with overflow chaining for any bucket that exceeds capacity.

Aspect	Static Hashing	Extendible Hashing
Structure	Fixed number of buckets, set once at design time	Directory of pointers to buckets; directory can double when needed
Best suited when	Data size is known and stable/slow-growing (matches this HR case)	Data size is unpredictable or grows/shrinks rapidly
Overflow handling	Chaining once load factor is exceeded — degrades gracefully but chains lengthen over time	Directory doubling + bucket splitting avoids long chains as data grows
Reorganization cost	None needed if provisioned correctly for 5–10 years of growth	None ever needed — grows incrementally, but adds directory-lookup overhead (1 extra pointer dereference) on every access
Implementation complexity	Simple — one hash function, fixed array of buckets	More complex — must maintain global/local depth and directory

Recommendation & Justification:

For this specific scenario, static hashing is the more appropriate and simpler choice. An HR system's headcount is known within a fairly narrow, predictable range (unlike, e.g., a viral social-media app), so the design can safely provision ~40% headroom above current headcount (7,000 slots for 5,000 employees) and expect years of stable performance without reorganization. Extendible hashing would only be justified if the company anticipated rapid, unpredictable workforce growth (e.g., aggressive expansion or M&A activity), where its ability to grow the directory incrementally — without ever needing a full, costly re-hash of all 5,000+ records — would outweigh its added directory-indirection overhead and implementation complexity.

Question 6: Combined Index Structure for an Airline Reservation System

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Problem Understanding:

Two distinct access patterns must both be served efficiently: (1) an exact-match point query ('show flight AI202') and (2) a range query on date ('show all flights departing between two dates', or 'show all instances of flight AI202 over the next month' — a combined equality + range predicate).

Proposed Combined Structure:

- Primary structure: a composite (concatenated-key) B+ Tree index on (flight_number, departure_date). Because flight_number is the leading key, an equality search on flight_number alone resolves in O(log n) by descending the tree to that key prefix; adding a date range then simply walks the linked leaves within that prefix — covering the very common 'this flight, over a date range' query with a single index traversal.

Justification:

Making flight_number the leading column of the composite B+ Tree index means the very common combined query — a specific flight across a date range — is answered by ONE index descent plus a linear leaf-chain walk, with no separate merge/intersection step required. The secondary date-only index exists specifically because a composite (flight_number, date) index cannot efficiently answer 'all flights on a given date' (that would require scanning across many different flight_number prefixes). This two-index combination — one composite index optimized for the flight+date case and one single-column index optimized for the date-wide case — covers both stated requirements without forcing either query pattern into a full table scan.

Question 7: File Organization & Secondary Indexing for a University Student Database

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Problem Understanding:

Typical university queries are department-scoped ('list all Computer Science students') and CGPA-scoped ('all students with CGPA above 8.5', for merit lists/scholarships) — often combined ('CS students with CGPA > 8.5'). department has low cardinality (a few dozen values) while CGPA is a continuous, range-queried attribute.

Recommended Design:

- Primary file organization: a clustered file physically partitioned/grouped by department (a clustering index on department). Since department has few distinct values, all records for the same department are stored in contiguous blocks — a 'list all CS students' query then touches only that department's block range, not the whole file.

Justification:

Clustering on department (rather than CGPA) is the right primary choice because department is used for equality filtering with low cardinality — clustering works best when it groups records that are frequently retrieved together, which favors an equality-queried, low-cardinality attribute over a continuously-valued, range-queried one like CGPA. CGPA, by contrast, is far better served by a separate non-clustering B+ Tree secondary index, since range queries need the sorted-key traversal a B+ Tree naturally supports, and CGPA lookups are typically institution-wide (merit lists) rather than always scoped to one department. This division of roles — clustering for the frequent equality dimension, indexing for the frequent range dimension — is the standard approach for supporting both query types without wasteful full scans.

Question 8: Disk Block Utilization Analysis: Heap vs. Sorted vs. Hashed

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Given Parameters:

Number of records (n) = 100,000 Record size = 50 bytes Block size = 4,096 bytes (4 KB)

Blocking factor (bf) = ⌊4096 / 50⌋ = 81 records/block (81 × 50 = 4,050 bytes used per full block; 46 bytes of internal fragmentation per block)

Total data volume = 100,000 × 50 = 5,000,000 bytes

Heap File (append-only, tightly packed):

Blocks required = ⌈100,000 / 81⌉ = 1,235 blocks (last block holds 100,000 − 1,234×81 = 46 records). Total space allocated = 1,235 × 4,096 = 5,058,560 bytes.

Utilization = 5,000,000 / 5,058,560 ≈ 98.8% — very high, since records are packed back-to-back with no reserved free space; the only waste is the ~46-byte per-block remainder that doesn't fit an extra record.

Sorted File (order must be maintained during inserts):

An initial bulk load achieves the same ~98.8% packing as the heap file. However, because the file must stay sorted, ongoing inserts typically require either (a) shifting many records — impractical at scale — or (b) reserving overflow/free space within blocks so new records can be inserted in the correct position without a full rewrite. In practice, sorted files intended for regular inserts are maintained at 60–70% initial fill factor to leave room for growth, giving an EFFECTIVE utilization of only ≈ 65% immediately after loading (rising back toward ~98% only after a full reorganization).

Hashed File (space reserved to keep collisions low):

Good hashing practice keeps the load factor around 70–75% to minimize collision chaining. Effective capacity per block = 0.75 × 81 ≈ 60 records. Blocks required = ⌈100,000 / 60⌉ = 1,667 blocks. Total space allocated = 1,667 × 4,096 = 6,829,568 bytes.

Utilization = 5,000,000 / 6,829,568 ≈ 73.2% — deliberately lower than the heap file, by design, to leave room in every bucket for future insertions without long overflow chains.

File Organization	Blocks Used	Space Allocated (bytes)	Utilization
Heap File	1,235	5,058,560	≈ 98.8%
Sorted File (with growth headroom)	≈ 1,900	≈ 7,782,400	≈ 64–65%
Hashed File (75% load factor)	1,667	6,829,568	≈ 73.2%

Justification / Trade-off:

Heap files achieve the highest raw space utilization because they simply pack records sequentially with no reserved slack — but this is only viable because heap files don't need free space for maintaining order or avoiding collisions. Sorted and hashed files intentionally sacrifice utilization (leaving 25–35% headroom) in exchange for algorithmic guarantees: sorted files need slack to insert without full rewrites, and hashed files need slack to keep collision chains short and preserve O(1) average access. This is a deliberate space-for-time

trade-off, not inefficiency — the 'wasted' space directly buys faster inserts (sorted) or faster lookups under growth (hashed).

Question 9: Composite Indexing Strategy for a Social Media Platform (500M Posts)

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Problem Understanding:

At 500 million posts, this is a write-heavy, massive-scale workload with three very different access patterns: (a) a user's own timeline — posts by one user_id, ordered by timestamp; (b) global/trending feeds — posts by timestamp across all users; and (c) hashtag search — all posts containing a given hashtag, typically also ordered by recency. A single index cannot efficiently serve all three; user_id and hashtag are both high-cardinality-adjacent but structurally different (one post = one author, but one post can have MANY hashtags — a many-to-many relationship).

Recommended Composite Indexing Strategy:

- Index 1 — Composite clustering index on (user_id, timestamp DESC): directly serves 'show this user's timeline' as a single index range-scan, with the most recent posts first. This is the primary/clustering structure since most read traffic in a social app is timeline-based.

Justification:

Treating hashtag as an inverted/posting-list index (rather than forcing it into the same composite B+ Tree as user_id) correctly models its many-to-many nature — a naive (user_id, hashtag, timestamp) composite index would force every hashtag lookup to scan across all users, since user_id is the leading key. Separating the three indexes by their actual query shape — user-scoped timeline, hashtag-scoped search, and global time-scoped feed — means each query touches only the index built for it, and each index can be independently partitioned and scaled to the write volume. The LSM-tree recommendation reflects that at 500 million (and growing) posts, insert throughput (new posts) matters as much as read latency, and LSM-trees convert random writes into sequential ones, which plain B+ Tree indexes cannot do as efficiently.

Question 10: Performance Comparison: Heap vs. Sorted vs. Hashed File for Supermarket Billing

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Problem Understanding:

A supermarket billing system continuously INSERTS new transaction records all day (every checkout) and occasionally SEARCHES for a specific transaction (e.g., a customer return/refund lookup by receipt number) or DELETES/archives old records. With 1,000,000 transactions/day, insert speed is the most operationally critical factor, but search speed matters for the (less frequent, but time-sensitive) refund/lookup workflow.

Assumptions:

n = 1,000,000 transaction records, 50-byte record, 4KB block ⇒ blocking factor = 81 records/block ⇒ ≈ 12,346 data blocks (same parameters as Q8, extended to 1 million records).

Operation	Heap File	Sorted File	Hashed File
Insert	O(1) — append to end. ≈ 1 I/O. Ideal for continuous real-time billing.	O(n) — must locate correct sorted position and shift records. ≈ 6,173 I/Os (avg.). Impractical for real-time inserts.	O(1) avg. — hash directly to target bucket. ≈ 1–2 I/Os. Excellent for real-time billing.
Search	O(n) — no ordering, must scan. ≈ 6,173 I/Os (avg.). Very poor for refund lookups.	O(log n) — binary search over sorted blocks. ≈ 14 I/Os ($\log_2$ 12,346). Excellent.	O(1) avg. — direct bucket access via hash. ≈ 1–2 I/Os. Excellent.
Delete	O(n) — find record (scan) + mark deleted. ≈ 6,173 I/Os. Poor.	O(n) — find (fast) but must shift records to close the gap. ≈ 6,173 I/Os (avg., dominated by shifting). Poor.	O(1) avg. — find bucket, remove/chain-unlink. ≈ 1–2 I/Os. Excellent.

Overall Recommendation: Hashed File Organization

For this supermarket billing scenario, a hashed file organization (e.g., a hash index on receipt/transaction_id, potentially extendible hashing to absorb daily/seasonal volume growth gracefully) delivers the best all-round performance: near-constant-time insert, search, AND delete — the only organization that is fast at all three operations simultaneously. The heap file is only competitive for inserts (which matters, but not exclusively), and the sorted file is only competitive for search — neither can keep up with the combined insert-heavy + occasional-lookup nature of live billing. Hashing's one drawback — no support for range queries (e.g., 'all transactions between 2pm and 4pm' for shift reconciliation) — can be separately addressed by maintaining a secondary, less frequently used B+ Tree index on transaction_timestamp purely for reporting purposes, without slowing down the primary hash-based billing path.